BHV_StationKeep

Station-keeping behavior.

Included MIT MOOS-IvP PDF sources:

- bhv_stationkeep.pdf

The StationKeep Behavior

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/bhvdocs/bhv_stationkeep`

1 The StationKeep Behavior **1**
1.1 Configuration Parameters . . . 2
1.2 The `station_pt` and `center_activate` Parameters . . . 4
1.3 The `swing_time` Parameter . . . 4
1.4 The `inner_radius`, `outer_radius`, and `outer_speed` Parameters . . . 4
1.5 Passive Low-Energy Station Keeping Mode . . . 5
1.6 Station Keeping On Demand . . . 7
1.7 The `visual_hints` Parameter . . . 8

Contents

1 The StationKeep Behavior

This behavior is designed to keep the vehicle at a given lat/lon or x,y station-keep position by varying the speed to the station point as a linear function of its distance to the point. The parameters allow one to choose the two distances between which the speed varies linearly, the range of linear speeds, and a default transit speed if the vehicle is outside the outer radius.

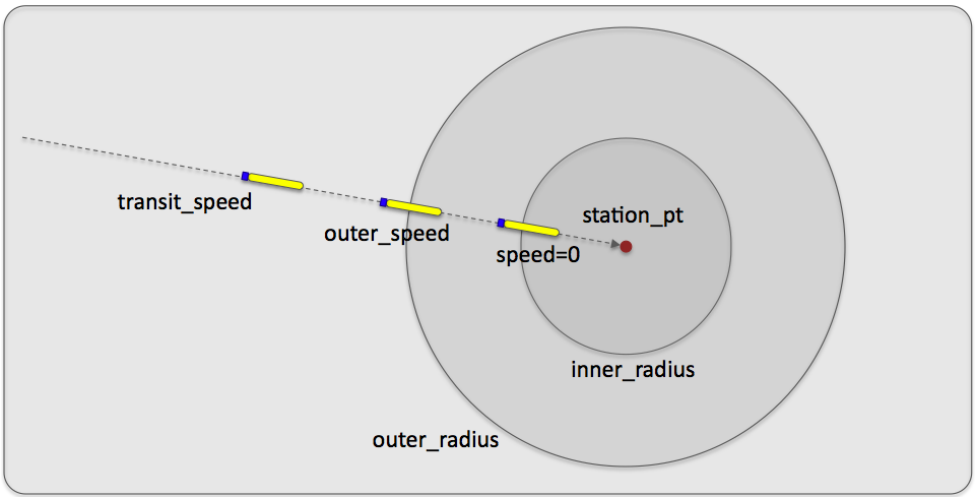

Figure 1: **The station-keep behavior parameters:** The station-keep behavior can be configured to approach the outer station circle with a given transit speed, and will decrease its preference for speed linearly between the outer radius and inner radius. The preferred speed is zero when the vehicle is at or inside the inner radius.

An alternative to this station keeping behavior is an active loiter around a very tight polygon with the `Loiter` behavior. This station keeping behavior conserves energy and aims to minimize propulsor use. The behavior can be configured to station-keep at a pre-set point, or wherever the vehicle happens to be when the behavior transitions into an active state.

The station-keep behavior was initially developed for use on an autonomous kayak. It's worth pointing out that a vehicle's control system, i.e., the front-seat driver described in Section **??**, may have a native station-keeping mode, in which case the activation of this behavior would be replaced by a message from the backseat autonomy system to invoke the station-keeping mode. It's also worth pointing out that most UUVs are positively buoyant and will simply come to the surface if commanded with a zero-speed.

1.1 Configuration Parameters

Listing 1.1: Configuration Parameters Common to All Behaviors.

`activeflag`:	A MOOS variable-value pair posted when the behavior is in the *active* state. [more].
`condition`:	Specifies a condition that must be met for the behavior to be running. [more].
`duration`:	Time in behavior will remain running before declaring completion. [more].
`duration_idle_decay`:	When true, duration clock is running even when in the *idle* state. [more].
`duration_reset`:	A variable-pair such as `MY_RESET=true`, that will trigger a duration reset. [more].
`duration_status`:	The name of a MOOS variable to which the vehicle duration status is published. [more].
`endflag`:	A MOOS variable-value pair posted when the behavior has completed. [more].
`idleflag`:	A MOOS variable-value pair posted when the behavior is in the *idle* state. [more].
`inactiveflag`:	A MOOS variable-value posted when the behavior is *not* in the *active* state. [more].
`name`:	The (unique) name of the behavior. [more].
`nostarve`:	Allows a behavior to assert a maximum staleness for a MOOS variable. [more].
`perpetual`:	If true allows the behavior to to run even after it has completed. [more].
`post_mapping`:	Re-direct behavior output normally to one MOOS variable to another instead. [more].
`priority`:	The priority weight of the behavior. [more].
`pwt`:	Same as `priority`.
`runflag`:	A MOOS variable and a value posted when a behavior has met its conditions. [more].
`spawnflag`:	A MOOS variable and a value posted when a behavior is spawned. [more].
`spawnxflag`:	A MOOS variable and a value posted when a behavior is spawned. [more].

templating: Turns a behavior into a template for spawning behaviors dynamically. [more].

updates: A MOOS variable from which behavior parameter updates are read dynamically. [more].

Listing 1.2: Configuration Parameters for the StationKeep Behavior.

Parameter	Description
center_activate	If true, station-keep at the vehicle's present position upon activation. Section 1.2.
hibernation_radius	A radius used for low-power, passive station-keeping. Section 1.5.
inner_radius	Distance to station-point within which the preferred speed is zero. Section 1.4.
outer_radius	Distance within which the preferred speed begins to decrease. Section 1.4.
outer_speed	Preferred speed at outer radius, decreasing toward inner radius. Section 1.4.
station_pt	An x,y pair given as a point in local coordinates. Section 1.2.
swing_time	Duration of drift of station circle with vehicle upon activation. Section 1.3.
transit_speed	Preferred speed beyond the outer radius. Section 1.4.
visual_hints	Preferences for rendering visual artifacts produced by the behavior. Section 1.7.

Listing 1.3: Example Configuration Block.

```
Behavior = BHV_StationKeep
{
  // General Behavior Parameters
  // ---------------------------
  name         = station-keep          // example
  pwt          = 100                   // default
  condition    = MODE==SKEEPING        // example
  inactiveflag = STATIONING = false    // example
  activeflag   = STATIONING = true     // example


  // Parameters specific to this behavior
  // ------------------------------------
     center_activate = false   // default
  hibernation_radius = -1      // default
        inner_radius = 4       // default
        outer_radius = 15      // default
        outer_speed  = 1.2     // default
      transit_speed  = 2.5     // default
          station_pt = 0,0     // default
          swing_time = 0       // default

        visual_hints = vertex_size  = 1           // default
        visual_hints = edge_color   = light_blue  // default
        visual_hints = edge_size    = 1           // default
        visual_hints = label_color  = white       // default
        visual_hints = vertex_color = red         // default
}
```

1.2 The station_pt and center_activate Parameters

The station-keep point is set in one of two ways: either with a pre-specified fixed position, or with the vehicle's current position when the vehicle transitions into the running state. To set a fixed station-keep position:

```
station_pt = 100,250
```

To configure the behavior to station-keep at the vehicle's current position when it enters the running state:

```
center_activate = true  // "true" is case insensitive
```

1.3 The swing_time Parameter

At the outset of station-keeping via center_activate, the vehicle typically is moving at some speed. Despite the fact that station-keeping is immediately active and typically results in a desired speed of zero if no other behaviors are active, the vehicle will continue some distance before coming to a near or complete stop in the water, thus "over-shooting" the station-keep point. This often means that the station-keep behavior will immediately turn the vehicle around to come back to the station-keep point. This can be countered by setting the behavior's swing_time parameter, the amount of time after initial center-activation that the station-keep point is allowed to drift with the current position of the vehicle before becoming fixed. The format is:

```
swing_time = <time-duration>   // default is 0 seconds
```

The time duration is given in seconds and should be in the range [0, 60]. If found to be outside this range it it simply clipped to the boundary value.

If the behavior enters the running state, but center-activation is not set to true, and no pre-specified fixed position is given, the behavior will not produce an objective function. It will remain in the running state, but not the active state. (Section **??** discusses run states.) In this situation, a warning will be posted via the helm appcast structure and by posting to the BHV_WARNING variable. In both cases the warning will read: "STATION_POINT_NOT_SET".

1.4 The inner_radius, outer_radius, and outer_speed Parameters

The inner_radius and outer_radius parameters affect the preferred speed of the behavior as it relates to the vehicle's current range to the station point. The preferred speed at the outer radius is given by the parameter outer_speed. The preferred speed decreases linearly to zero as the vehicle approaches the inner radius. The default values for the inner and outer radii are 4 and 15 respectively. If configured with values such that the inner is greater than the outer, this will not trigger an error, but the two radii parameters will be collapsed to the value of the inner radius on the first iteration of the behavior. The transit_speed parameter indicates the desired speed when the vehicle is outside the outer_radius. The default value for transit_speed is 2.5 meters per second. If the outer_speed is set higher than the transit_speed the transit_speed will automatically be raised to the outer_speed.

1.5 Passive Low-Energy Station Keeping Mode

The station-keep behavior can be configured to operate in a ”passive” mode. This mode differs from the default mode primarily in the way it acts after it reaches the inner-radius, i.e., the point at which the behavior regards the vehicle to be on-station and outputs a preferred speed of zero. In the normal mode, the behavior will begin to output a preferred heading and non-zero speed as soon as the vehicle slips beyond the inner-radius. In the passive mode, the behavior will let the vehicle drift or otherwise move to a distance specified by the `hibernation_radius` before it resumes outputting a preferred heading and non-zero speed. The idea is shown in Figure 2.

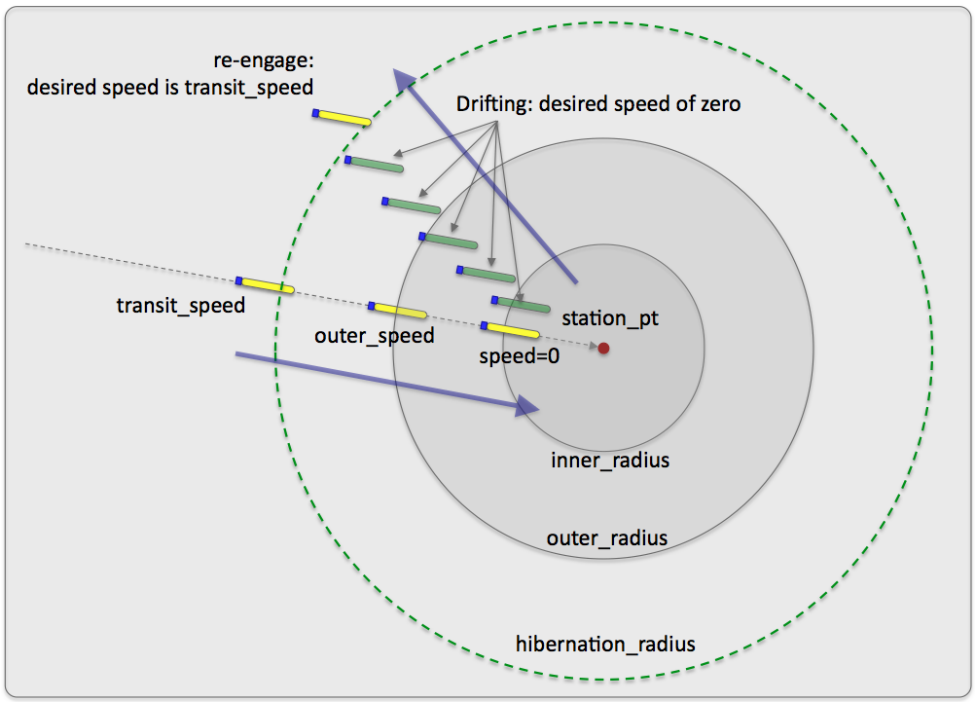

Figure 2: **Passive station-keeping:** The station-keep behavior can be configured in the ”passive” mode. The vehicle will move toward the station point until it reaches the `inner_radius` or until progress ceases. It will then drift until its distance to the station point is beyond the `hibernation_radius`. At this point it will re-engage to reach the station-point and may trigger another behavior to dive.

This mode was built with UUVs in mind. Most UUVs are deployed having a positive buoyancy (battery dies - vehicle floats to the surface). They need to be moving at some speed to maintain a depth. Furthermore, it may not be safe to assume that a UUV can effectively execute a desired heading when it is operating on the surface. For these reasons, when operating in the passive mode, this behavior will publish a variable indicating whether it is in the mode of drifting or attempting to make progress toward the station point. The status is published in the variable `PSKEEP_MODE`, short for ”passive station-keeping mode”. This variable will be set to `"SEEKING_STATION"` when outputting a non-zero speed preference, and presumably moving toward the station-point. The variable will be set to `"HIBERNATING"` otherwise. This opens the option of configuring the helm with the ConstantDepth behavior to work in conjunction with the StationKeep behavior by conditioning

the ConstantDepth behavior to be running only when PSKEEP_MODE="SEEKING_STATION". The idea is shown in Figure 3.

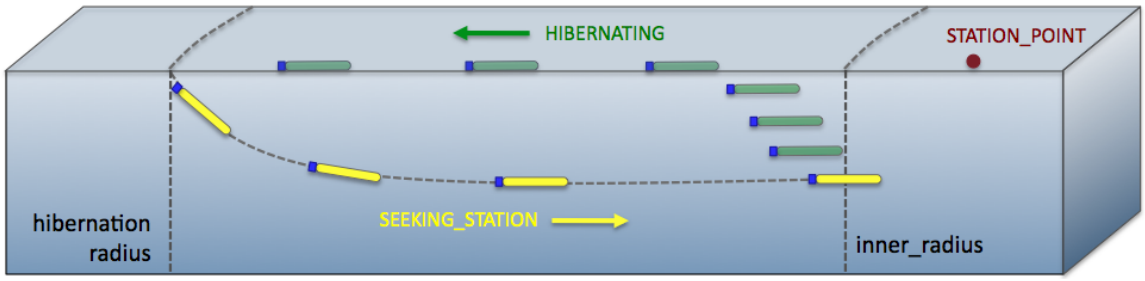

Figure 3: **Passive station-keeping with depth coordination:** The passive mode can be coordinated with the ConstantDepth behavior to dive each time the StationKeep behavior enters the "SEEKING_STATION" mode. This ensures that a UUV needing to be at depth to have reliable heading control will indeed be at depth when it needs to be.

This behavior mode is regarded as "low-power" due to the presumably long periods of drifting before resuming actively seeking the station point. A couple of safeguards are designed to ensure that when the behavior is in the "STATION_SEEKING" mode, that it does not get hung or stuck in this mode for much longer than intended or needed. How could one become stuck in this mode? Two ways - by either reaching an equilibrium at-speed, (and perhaps at-depth) state where the vehicle is neither progressing toward or way from the inner_radius, or by repeatedly "missing" the inner_radius by heading right past it.

Both cases can be guarded against and detected by monitoring the history of vehicle speed in the direction of the station-point. If this speed becomes zero, an equilibrium state is assumed, and if it becomes negative, it is assumed that the vehicle missed the inner radius circle entirely. In short, the StationKeep behavior exits the "STATION_SEEKING" mode and enters the "HIBERNATING" mode when it detects the vehicle speed toward the station-point reach zero. To calculate this vehicle speed, a ten-second history of range to the station-point is kept by the behavior. A zero speed, or "stale-progress" criteria is declared simply if the range to the station-point for the most recent measure in the history is not less than the range of ten seconds ago in the history list. The behavior will transition into the "HIBERNATING" mode if either the inner-radius or stale-progress criteria are met.

It is also possible that when the StationKeep behavior enters the "SEEKING_STATION" mode from the "HIBERNATING" mode, that the vehicle initially begins to open its range to the station-point before it begins to close range. This would be expected, for example, if the vehicle were pointed away from the station-point when the behavior first entered the "SEEKING_STATION" mode. In this case it's quite possible that the behavior would correctly, but unwantingly, infer that the stale-progress criteria has been met. For this reason, the stale-progress criteria is not applied until an "initial-progress" criteria is met after entering the "SEEKING_STATION" mode. The same ten second history is used to detect when the vehicle begins to make initial progress, i.e., closing range, toward the station-point.

1.6 Station Keeping On Demand

A common, and perhaps recommended configuration, is to have one station-keep behavior defined for a given helm configuration and have it set to be usable in one of three ways: (a) station-keep at a default pre-specified position, (b) station-keep at a specified position dynamically provided, or (c) station-keep at the vehicle's present position when activated. The behavior would be configured as follows:

```
station_pt      = 100,200 // The default station-keep point
center_activate = false
updates         = STATION_UPDATES
condition       = STATION_REQUEST = true
```

Then, to use the station-keep behavior in the above three ways, the following three pairs of postings, i.e., pokes, to the MOOSDB would be used. See Section **??** for more on the `updates` parameter defined for all behaviors - by utilizing this dynamic configuration hook, the one behavior configuration above can be used in these different manners. The first pair would result in the behavior keeping station at its pre-arranged point of (100,200):

```
STATION_REQUEST = true
STATION_UPDATES = center_activate=false
```

The second line above dynamically configures the behavior parameter `center_activate` to be false to ensure that the point given by the original `station_pt` parameter is used. Even though the `center_activate` parameter is initially set to false, the above usage sets it to false anyway, to be safe, and in case it has been dynamically set to true in a prior usage.

In the second case below, again the `center_activate` parameter is dynamically set to false for the same reasons. In this case the `station_point` parameter is also dynamically configured with a given point:

```
STATION_REQUEST = true
STATION_UPDATES = station_pt=45,-150 # center_activate=false
```

In the last case, below, the behavior is activated and configured to station-keep at the vehicle's present position when activated. There is no need to tinker with the `station_pt` parameter since this parameter is ignored when `center_activate` is true:

```
STATION_REQUEST = true
STATION_UPDATES = center_activate=true
```

It's worth noting that above variable-value pairs that trigger the StationKeep behavior could have come from a variety of sources. They could be endflags from another behavior. They could have come from a poke using `uPokeDB`, `uTimerScript`, `pMarineViewer` or any third party command and control interface.

1.7 The visual_hints Parameter

Although the primary output of the StationKeep behavior is an IvP Function, a number of visual properties are also published for convenience in mission monitoring. This includes (a) the inner_radius, (b) the outer_radius, and (c) the hibernation_radius if used. These visual artifacts have default properties in size and color that may be altered to the user's preferences. These preferences are configurable through the visual_hints parameter. Each parameter below is used in the following way by example:

```
visual_hint = vertex_size=3, edge_size=2
visual_hint = vertex_color=khaki
```

- edge_color: The color of edges rendered in the loiter polygon. The default is "white".
- edge_size: The width of edges rendered in the loiter polygon. The default is 1.
- label_color: The color of labels rendered with the innner and outer radii. The default is "gray50".
- vertex_color: The color of vertices rendered in the loiter polygon. The default is "dodger_blue".
- vertex_size: The size of vertices rendered in the loiter polygon. The default is 1.

Rendering of vertices may be shut off with a size of zero, and labels may be shut off with the special color "invisible". For a list of legal colors, see Appendix **??**.